Trabajo Práctico N°2
Algoritmos Genéticos

Agustin Alonso (63316)
Gonzalo Candisano (62616)
Lucas González Rouco (63366)
Timoteo Feeney (62742)

Ejercicio 2 - Aproximación de Imágenes con Triángulos

Representación

- **Locus:** Es la posición específica que ocupa un dato en la lista (el índice que define qué triángulo se dibuja primero y cuál va encima)
- **Alelo:** Es la unidad mínima de información, un valor individual (ej. x1, y1, r, g, etc)
- **Cromosoma:** Es un triángulo completo formado exactamente por la agrupación de 10 alelos (x1, y1, x2, y2, x3, y3, r, g, b, a)
- **Genotipo:** Es la estructura de datos que el motor realmente procesa, un arreglo compuesto por N cromosomas (la lista entera de triángulos)
- **Fenotipo:** Es la expresión visual de esa matemática, la imagen renderizada que se genera al dibujar esos triángulos sobre el canvas para calcular el error
- **Individuo:** Es el objeto completo que el algoritmo evalúa, agrupa en un solo paquete su Genotipo (datos), su Fenotipo (imagen) y su puntaje de Fitness.

Función de Aptitud

- **Métrica:** Inversa del Error Cuadrático Medio Normalizado

```python
def compute_fitness(individual: Individual, target: np.ndarray, width: int, height: int) -> float:
    rendered = np.array(render(individual, width, height).convert("RGB"), dtype=np.float32)
    mse = np.mean(((rendered - target) / 255.0) ** 2)
    return 1.0 - float(mse)
```

Función de Aptitud

- **Métrica:** Inversa del Error Absoluto Medio Normalizado

```
def compute_fitness(individual: Individual, target: np.ndarray, width: int, height: int) ->
floatrendered_img = render(individual, width, height).convert("RGB")
    rendered = np.array(rendered_img, dtype=np.float32)

    mae = np.mean(np.abs(rendered - target)) / 255.0

    return 1.0 - float(np.sqrt(mae))
```

Criterios de Parada

- **Objetivo Absoluto:** best.fitness >= 0.99
- **Estancamiento Térmico:** 150 generaciones
- **Límite Computacional:** 2000 generaciones

```python
if best.fitness >= cfg.get("target_fitness", 1.0):
    print("Criterio de parada: fitness objetivo alcanzado.")
    break
```

```python

for generation in range(1, cfg["max_generations"] + 1):
    parents = select(population, cfg["selection"]["k"])

    random.shuffle(parents)
    offspring = []
```

```python
if use_stagnation:
    if best.fitness > best_ever + stagnation_eps:
        best_ever = best.fitness
        gens_without_gain = 0
    else:
        gens_without_gain += 1
        if best.fitness > best_ever:
            best_ever = best.fitness
        if gens_without_gain >= stagnation_n:
            print(
                "Criterio de parada: estancamiento "
                f"({stagnation_n} generaciones sin mejora > {stagnation_eps})."
            )
            break
```

Métodos de Cruza

- **Cruza de 1 punto:**
  - Corta la lista en dos. Un padre hereda todo el fondo y el otro aporta los detalles superficiales.
- **Cruza de 2 Puntos:**
  - Intercambia un "bloque central" de triángulos. Permite heredar el fondo y el frente de un padre, inyectando una capa intermedia del otro.
- **Cruza Anular:**
  - Intercambia un bloque continuo que puede "dar la vuelta" al principio de la lista. Evita el sesgo geométrico de elegir siempre el centro.
- **Cruza Uniforme:**
  - Tira una moneda (50% de probabilidad) triángulo por triángulo. Rompe por completo el orden lógico de las capas.

Métodos de Cruza según el fin buscado

- **Para explorar rápido:** *Cruza Uniforme*
  - Durante las fases iniciales, necesitamos una alta tasa de recombinación para explorar el espacio de soluciones.
  - Romper el orden de los triángulos es ideal en esta etapa para encontrar los colores base y no quedarnos estancados en un fondo incorrecto.
- **Para explotar la imagen:** *Cruza de 1 Punto o Anular*
  - Cuando el fondo y la forma ya están bien armados, seguir mezclando al azar destruiría el progreso.
  - Estos métodos heredan bloques enteros (ej: "todo el fondo intacto"), protegiendo las estructuras visuales que ya funcionan y solo puliendo detalles.

Métodos de Mutación

- **Mutación Local** *(Explotación / Ajuste Fino):*
  - **Gene:** Altera 1 solo atributo de un triángulo al azar (dependiendo de *mutation_rate*)
  - **Multigene**: Altera un número fijo N de genes.
- **Mutación Global** *(Exploración / Disrupción):*
  - **Uniforme:** Reemplaza triángulos enteros por nuevos al azar (*mutation_rate)*.
  - **Completa:** Muta todos los triángulos del individuo a la vez.
- **Mutación Inteligente:**
  - **No Uniforme:** La magnitud del cambio disminuye a medida que avanzan las generaciones.

Resultados

Primera Imagen - Colombia

La bandera de Colombia tiene como particularidades el ser una imagen simétrica y no tener el color blanco.

Parámetros seleccionados

- **Método de selección:** *Torneos determinísticos de tamaño 5*
  - Buscamos una presión de selección que balancee entre elitismo y retención de diversidad. Evitamos selecciones completamente arbitrarias y elitistas.

- **Método de cruza:** *Uniforme con probabilidad de 0,5*
  - Maximizar la recombinación entre padres de alta calidad. Equilibra la exploración y no nos generó resultados destructivos.

- **Método de mutación:** *No Uniforme con tasa de 0,12 y b = 1*
  - Buscamos una mutación más agresiva en las primeras generaciones, con decaimiento para garantizar micro mutaciones en las últimas generaciones. El valor de tasa pareció una buena frecuencia de refinamiento, evitando que las mutaciones sean destructivas.

Primera prueba

Mantenemos **fijos** los parámetros relacionados a la **cruza** y la **mutación**, para **explorar métodos de selección**.

- Torneos determinísticos de tamaño 5
  - Nuestra hipótesis inicial, suponemos que es el método de selección que brindará mejores resultados.
- Elitista
  - Probamos selección elitista para evaluar realmente si es importante tener algun grado de variedad genética, y evitar el elitismo puro.
- Universal
  - Probamos un método de selección estocástico.

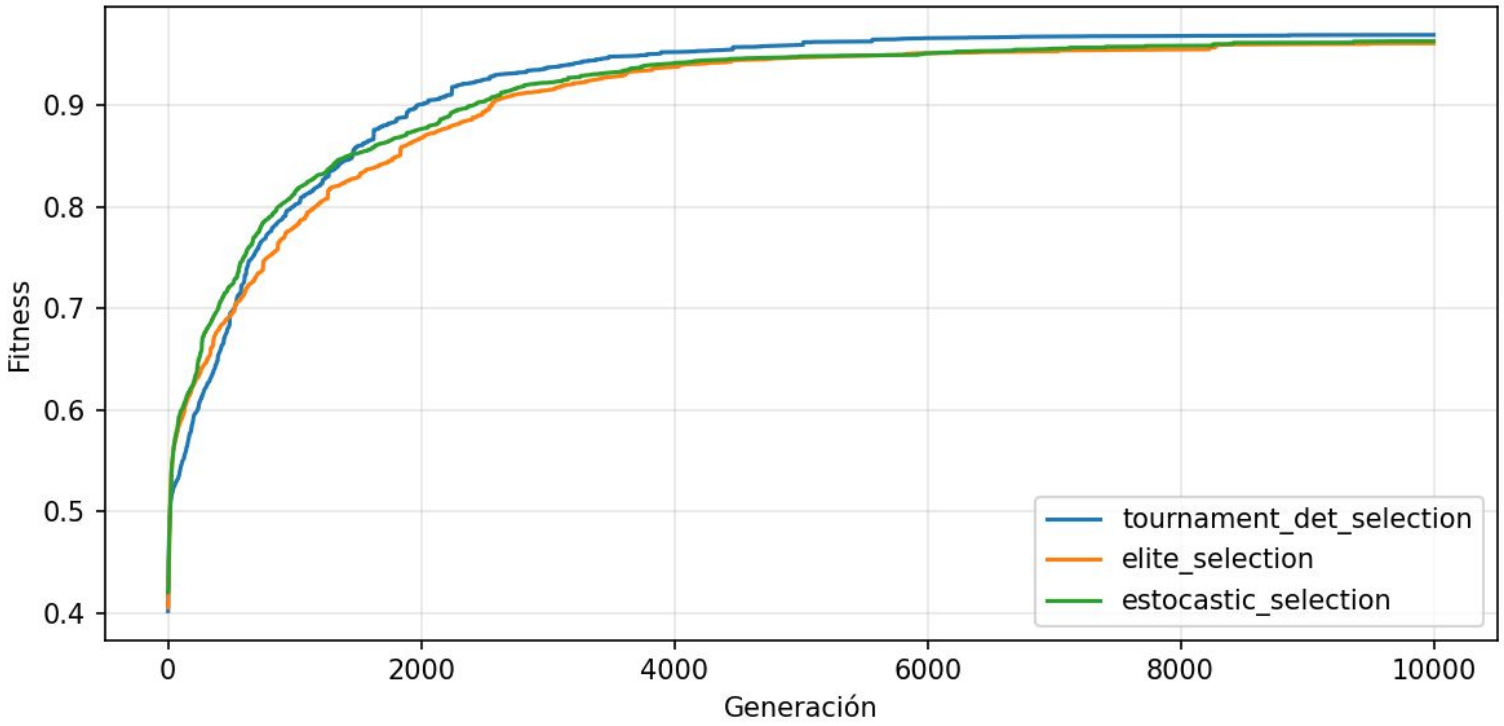
Resultados

Torneos Deterministicos

Elitista

Universal

Análisis - ¿Es la imagen muy simple?



Segunda imagen - Antigua y Barbuda

Con la bandera de Colombia obtuvimos resultados muy certeros que no nos dio mucha capacidad de análisis. Repetimos la misma prueba pero con una imagen más compleja, como lo puede ser la bandera de Antigua y Barbuda.

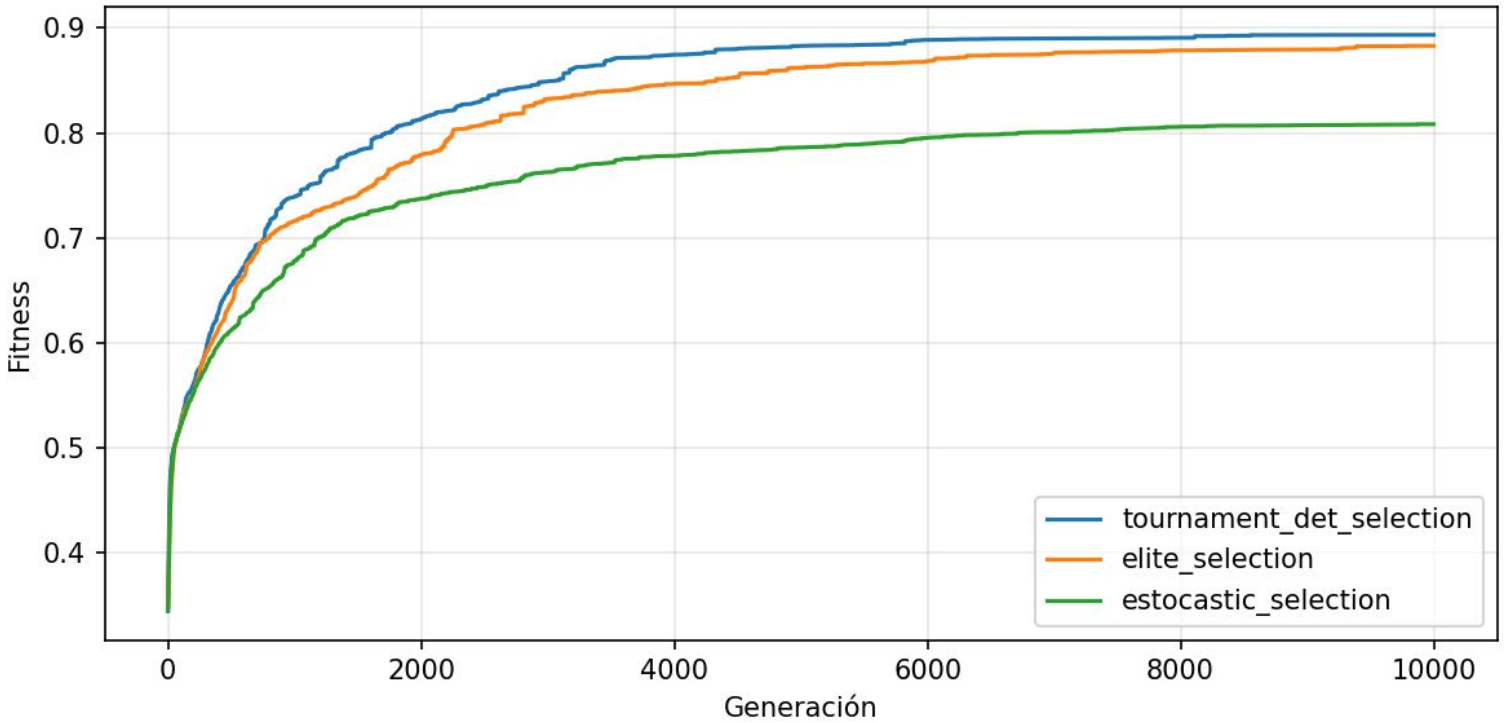
Resultados

Torneos Deterministicos

Elitista

Universal

Análisis


0.9
0.8
0.7
0.6
0.5
0.4
Fitness
0
2000
4000
6000
8000
10000
Generación
tournament_det_selection
elite_selection
estocastic_selection

Algunas conclusiones...

- Cuando la imagen es muy simple, para cualquier método de selección que empleamos el resultado es certero.
- Al complejizar la imagen a generar, obtenemos mayores diferencias entre los resultados.
- La selección mediante torneos determinísticos obtuvo el mejor resultado, destacando más en las porciones de la imagen menos geométricas. Estimamos que se debe a cómo la población converge a individuos de élite, pero de manera escalonada, lo que genera mayor diversidad genética para las porciones más complicadas de la imagen.
- La selección elitista tuvo dificultades en las partes menos geometricas. Suponemos que se debe a la falta de variación genética que puede generar priorizar siempre los individuos elitistas.

Siguiente prueba - ¿Como puede afectar la mutación?

Partiendo de que los datos que obtuvimos respecto al método de selección, veamos si variando el método de mutación podemos alcanzar un mejor resultado.

- No Uniforme
- Gen
- Multigen

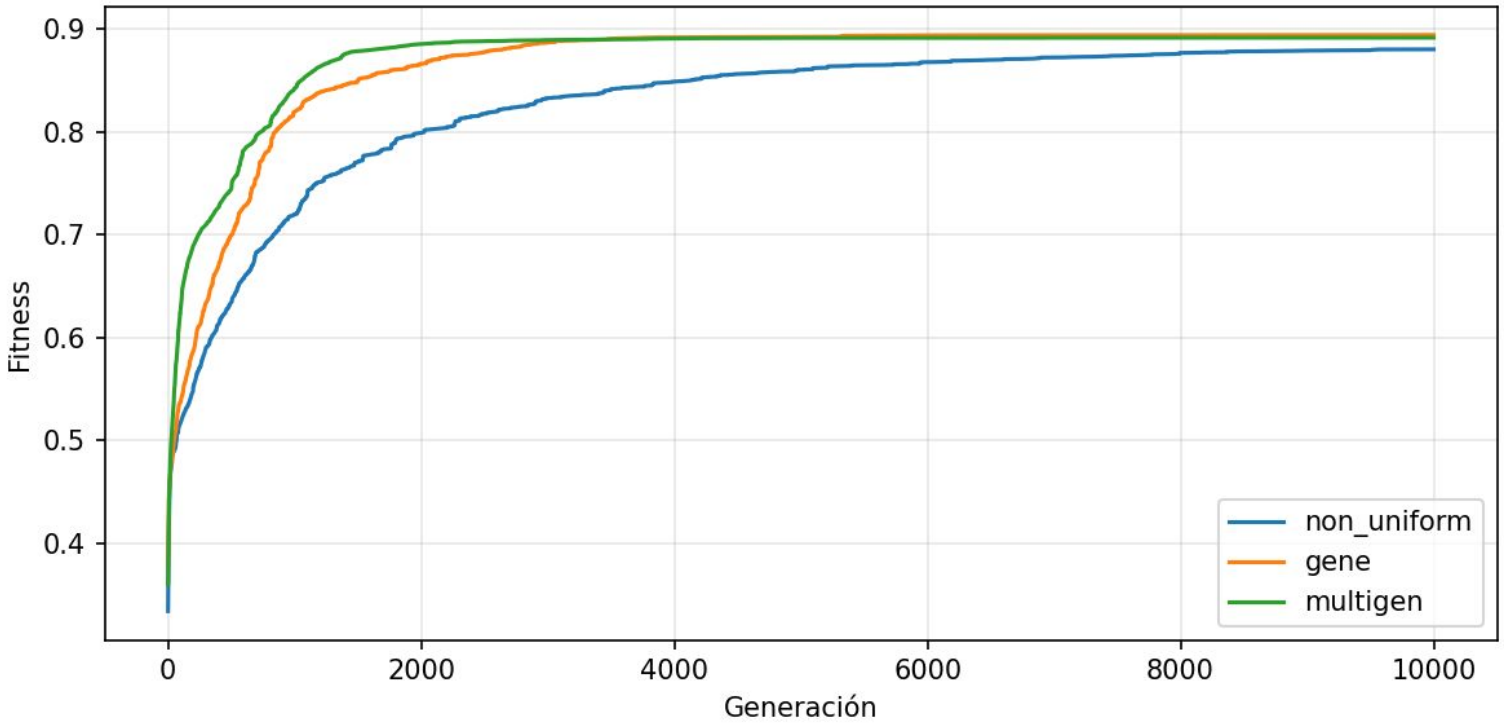
Resultados

No Uniforme

Elitista

Universal

Análisis


0.9
0.8
0.7
0.6
0.5
0.4
Fitness
0
2000
4000
6000
8000
10000
Generación
non_uniform
gene
multigen

Tercera imagen - Logo de Chrome

Para el logo de Chrome, nos propusimos probar primero diferentes perfiles. Algunos más conservadores y otros más disruptivos.

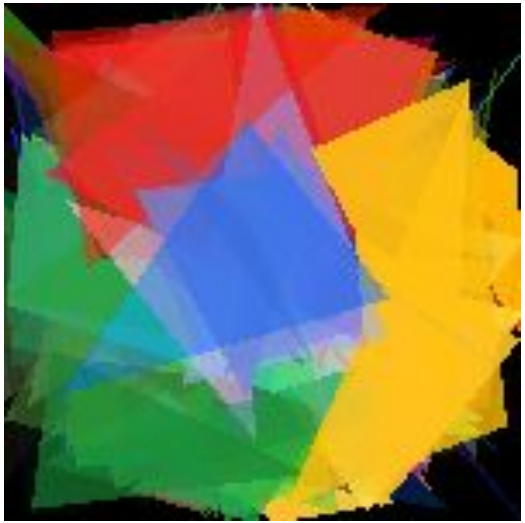
Perfiles seleccionados

- **Elitista:** Selección Élite + Mutación Gene (0.02) + Reemplazo Tradicional.
- **Balanceado:** Torneos (k=6) + Mutación No Uniforme (b=6) + Reemplazo Tradicional.
- **Estocástico:** Selección Universal + Mutación Uniforme (0.18) + Reemplazo Young Bias.

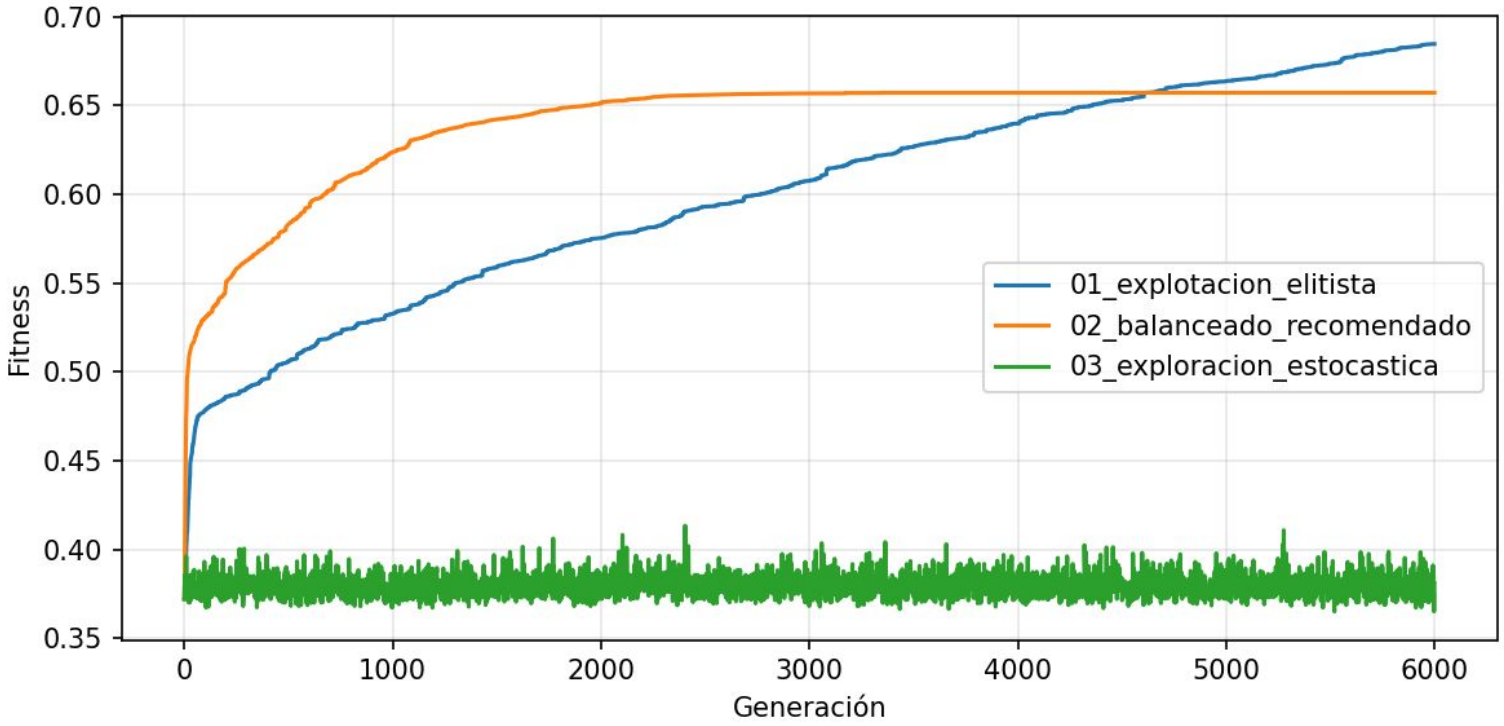
Todos con esto en común:

- Máximo de 250 primitivas (triángulos) y 6000 generaciones.
- Población de 200 individuos.

Resultados



Análisis

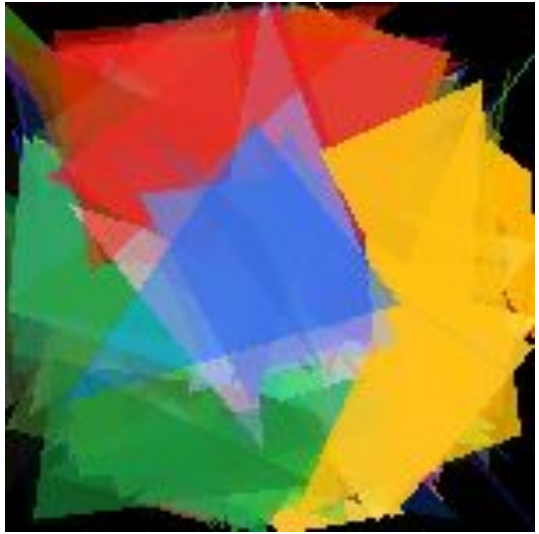

0.70
0.65
0.60
0.55
0.50
0.45
0.40
0.35
Fitness
0
1000
2000
3000
4000
5000
6000
Generación
01_explotacion_elitista
02_balanceado_recomendado
03_exploracion_estocastica

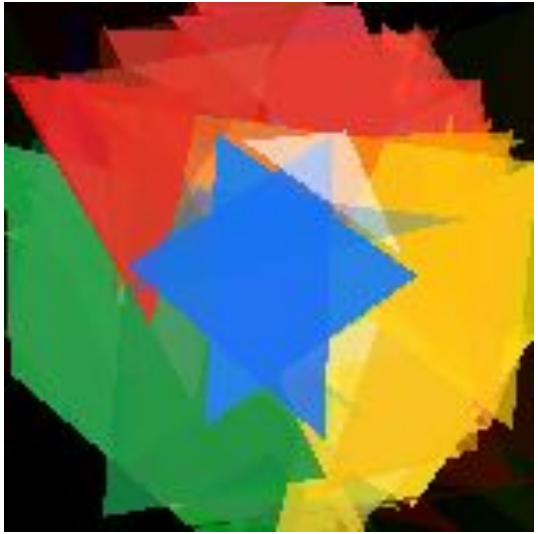
Rediseño de perfiles - Logo de Chrome

Una vez que tuvimos esos resultados, rediseñamos y modificamos los perfiles para seguir analizando

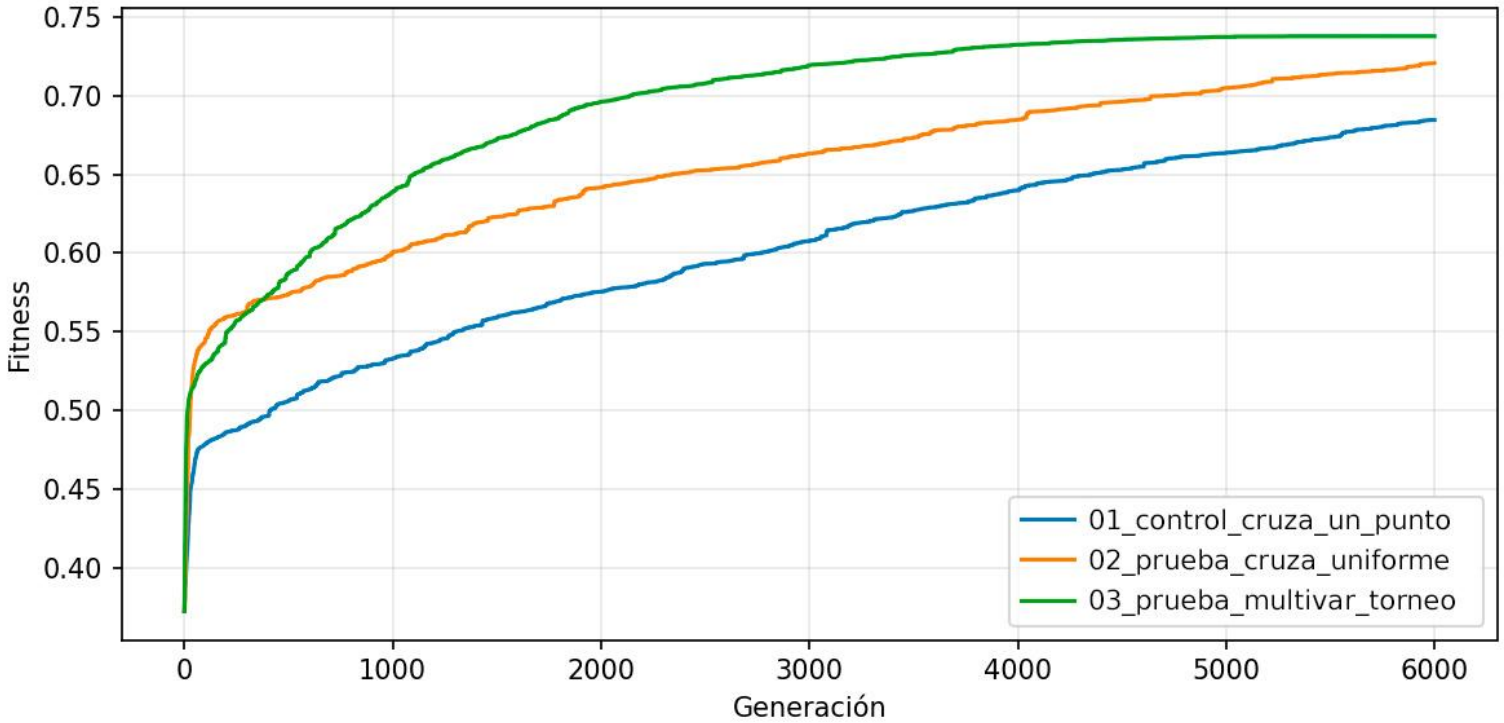
- **Control:** El ganador previo para tener una base
- **Prueba Uniforme:** Elitismo + Cruza Uniforme (buscando velocidad)
- **Prueba Multivariable:** Torneos + Mutación No Uniforme suave (b=2).

Resultados





Análisis

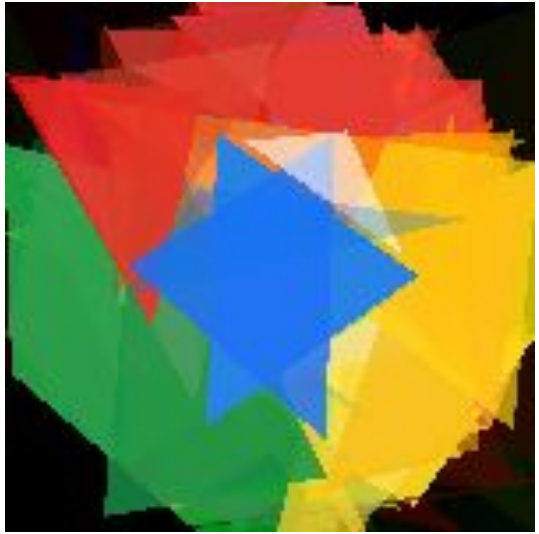

0.75
0.70
0.65
0.60
0.55
0.50
0.45
0.40
Fitness
0
1000
2000
3000
4000
5000
6000
Generación
01_control_cruza_un_punto
02_prueba_cruza_uniforme
03_prueba_multivar_torneo

Rediseño de perfiles #2 - Logo de Chrome

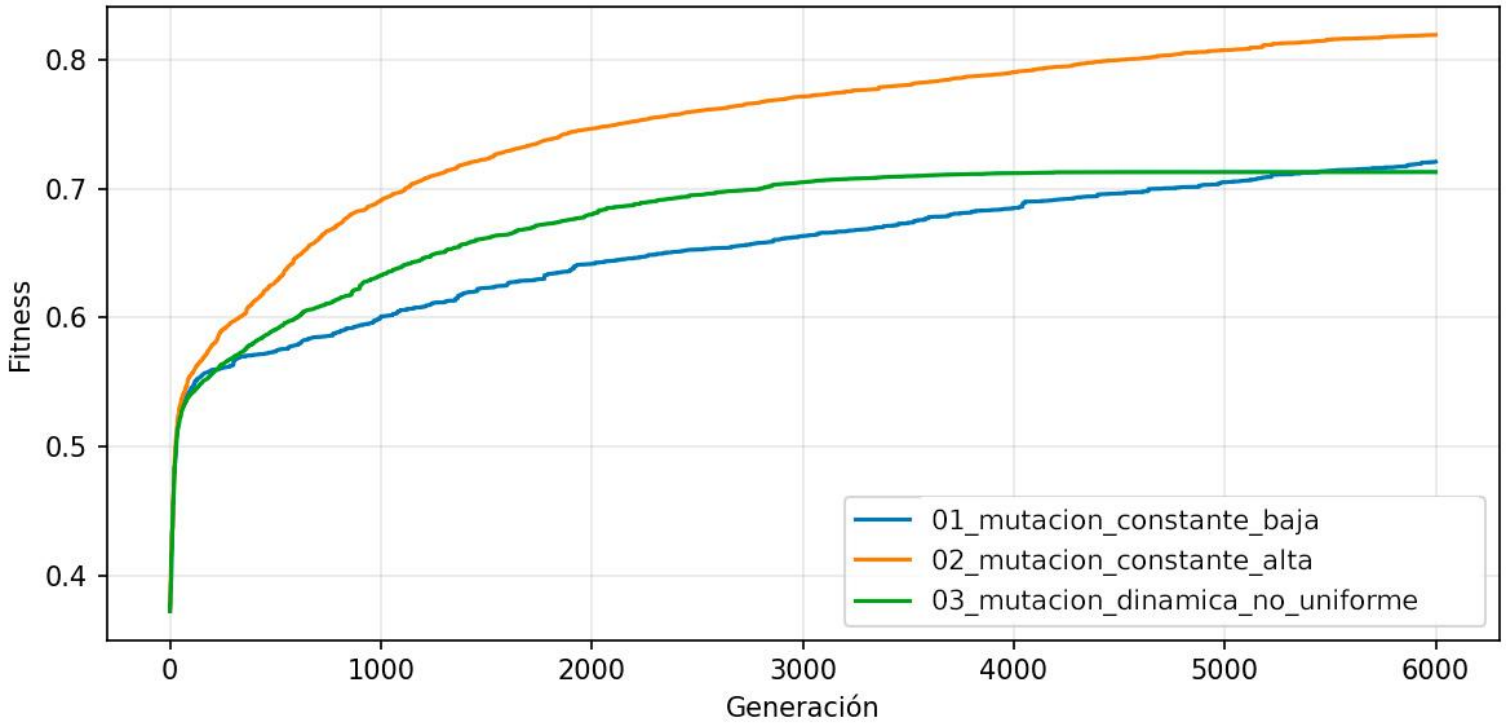
Con la base ganadora ya definida (Selección Elite y Cruza Uniforme), dedicamos este experimento a aislar y analizar exclusivamente el impacto de la mutación

- **Prueba Mutación Baja:** Mutación Constante al 2% (buscando explotar la convergencia rápida como nuestra nueva base).
- **Prueba Mutación Excesiva:** Mutación Constante al 15% (buscando forzar la exploración al máximo para ver el límite de rotura).
- **Prueba Mutación Dinámica:** Mutación No Uniforme con decaimiento b=3 (buscando el balance perfecto: exploración inicial y refinamiento final).

Resultados



Análisis

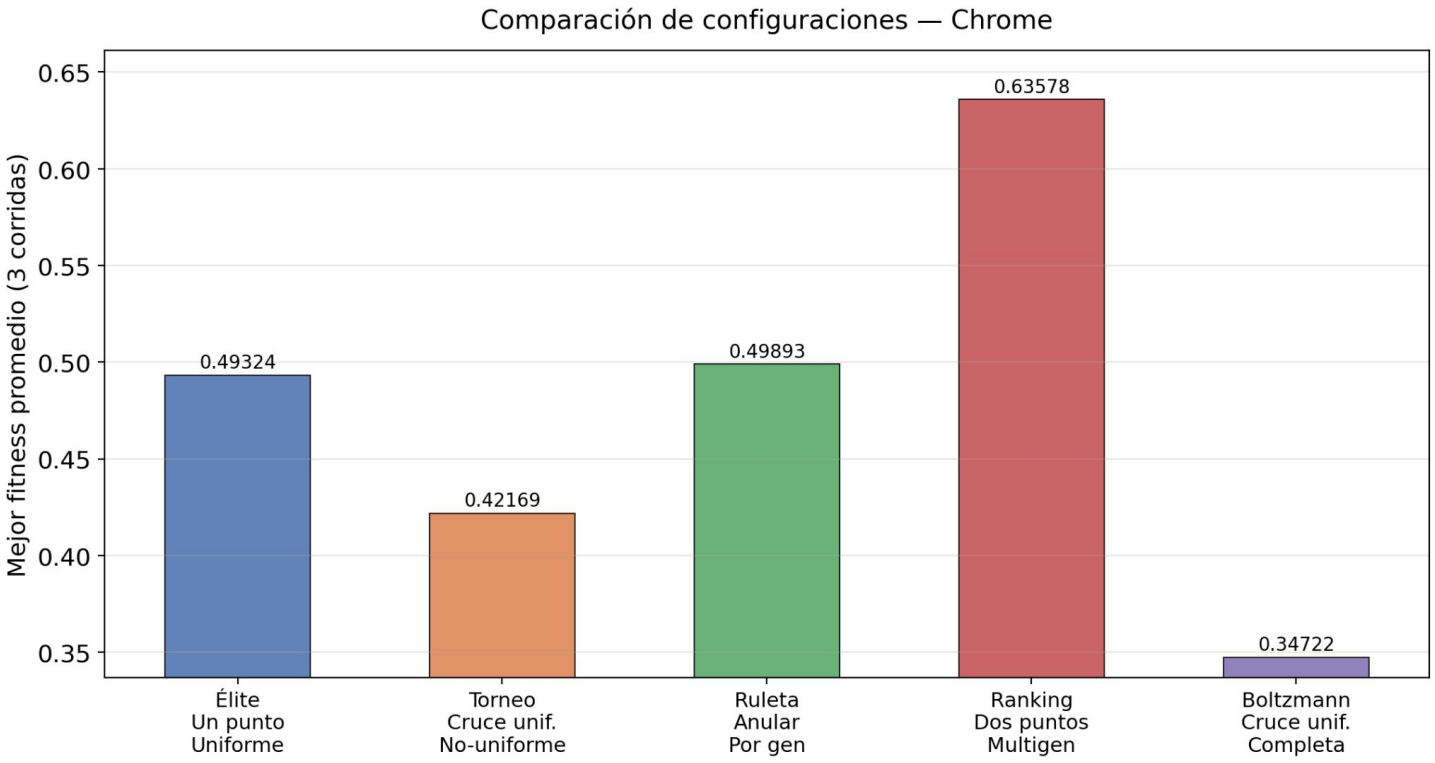

0.8
0.7
0.6
0.5
0.4
Fitness
0
1000
2000
3000
4000
5000
6000
Generación
01_mutacion_constante_baja
02_mutacion_constante_alta
03_mutacion_dinamica_no_uniforme

Independencia entre métodos

Objetivo

- **Decidir** si los distintos parámetros son independientes entre sí
- Se probaron 5 combinaciones distintas de:
  - selección
  - cruza
  - mutación
- Cada configuración se evaluó en múltiples corridas

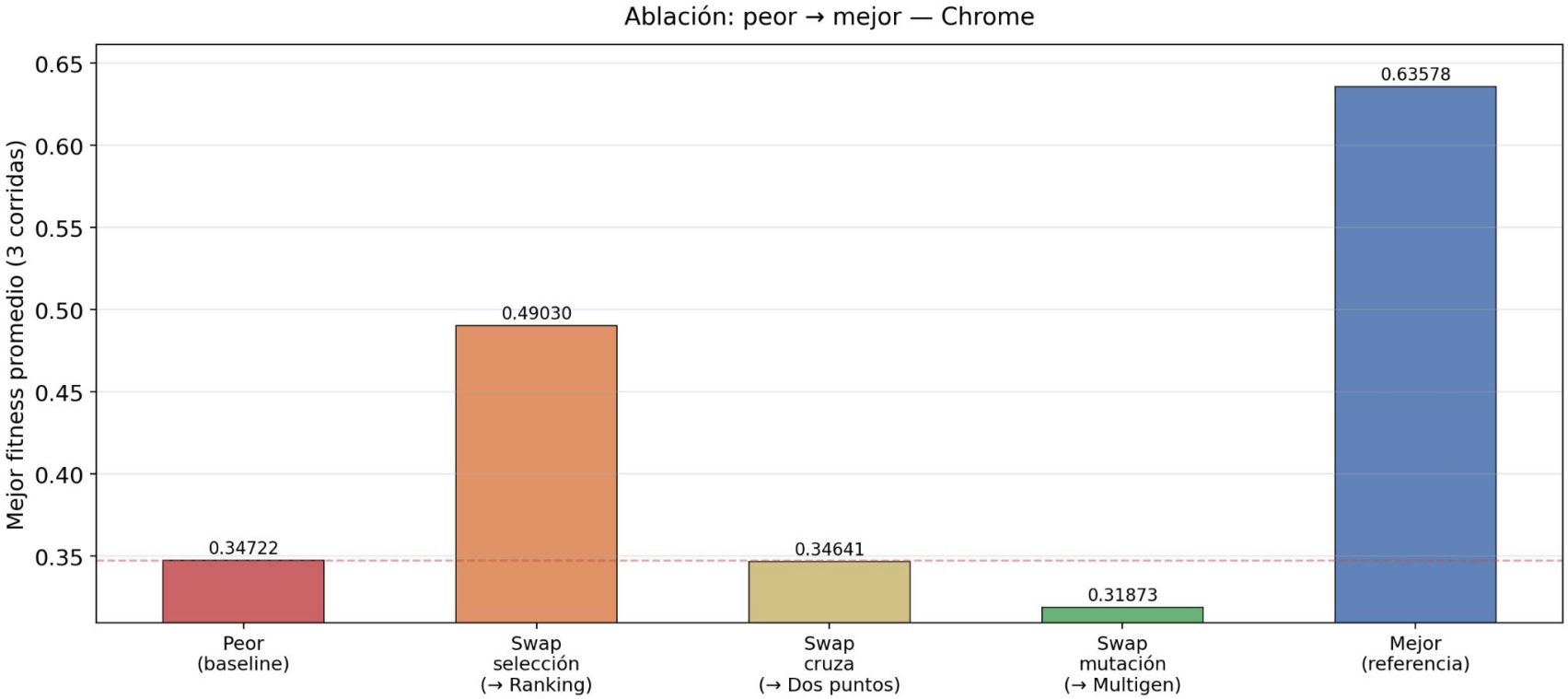
Resultados


Comparación de configuraciones — Chrome
Mejor fitness promedio (3 corridas)
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0.49324
0.42169
0.49893
0.63578
0.34722
Élite
Un punto
Uniforme
Torneo
Cruce unif.
No-uniforme
Ruleta
Anular
Por gen
Ranking
Dos puntos
Multigen
Boltzmann
Cruce unif.
Completa

Análisis

- Se identificaron:
  - Peor configuración
  - Mejor configuración
- Partimos de la peor configuración como baseline
- Reemplazamos un operador a la vez por el de la mejor config:

Resultados


Ablación: peor → mejor — Chrome
Mejor fitness promedio (3 corridas)
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0.34722
0.49030
0.34641
0.31873
0.63578
Peor
(baseline)
Swap
selección
(→ Ranking)
Swap
cruza
(→ Dos puntos)
Swap
mutación
(→ Multigen)
Mejor
(referencia)

Observaciones

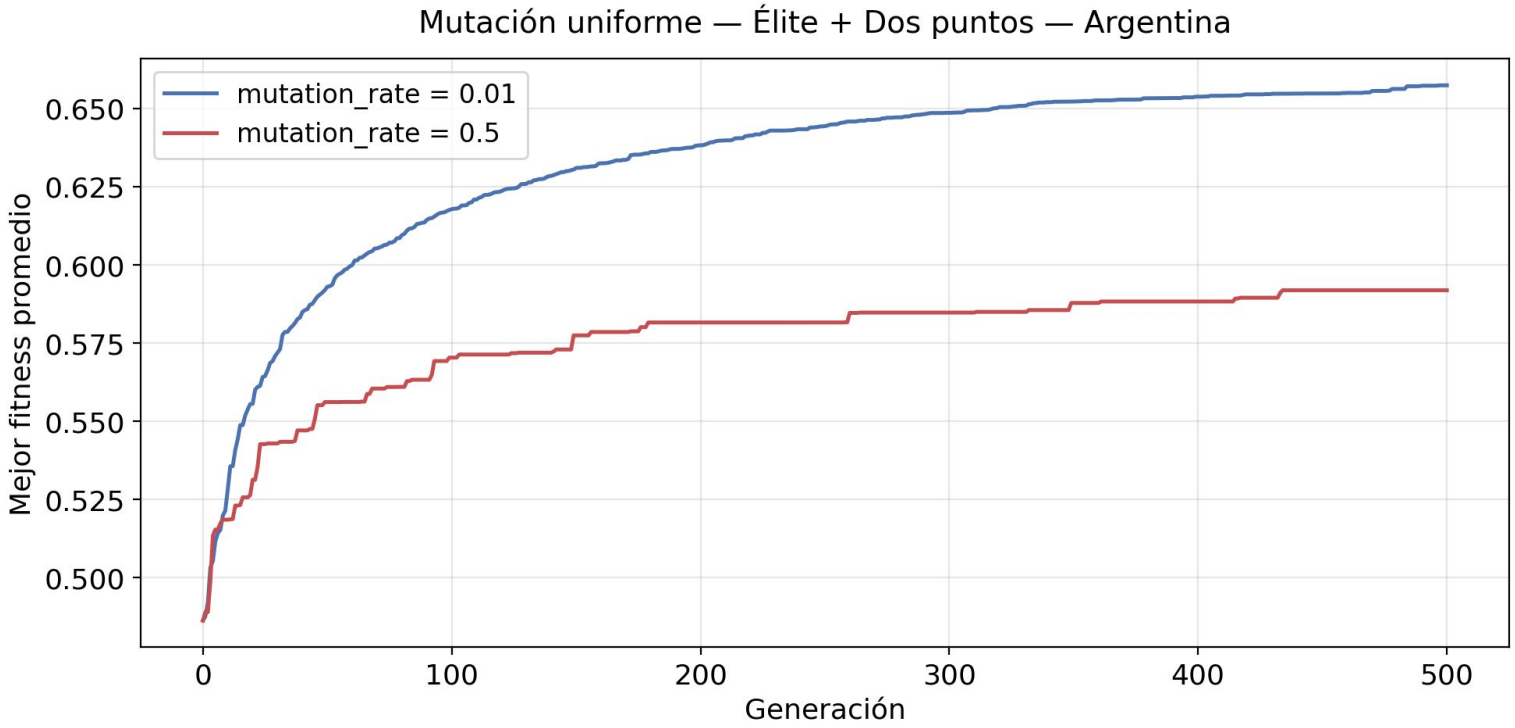
- Los efectos no son independientes
- Algunos operadores:
  - empeoran de forma aislada
  - mejoran en conjunto
- No alcanza con optimizar cada operador por separado
- Existen sinergias entre componentes

Probabilidad de Mutación

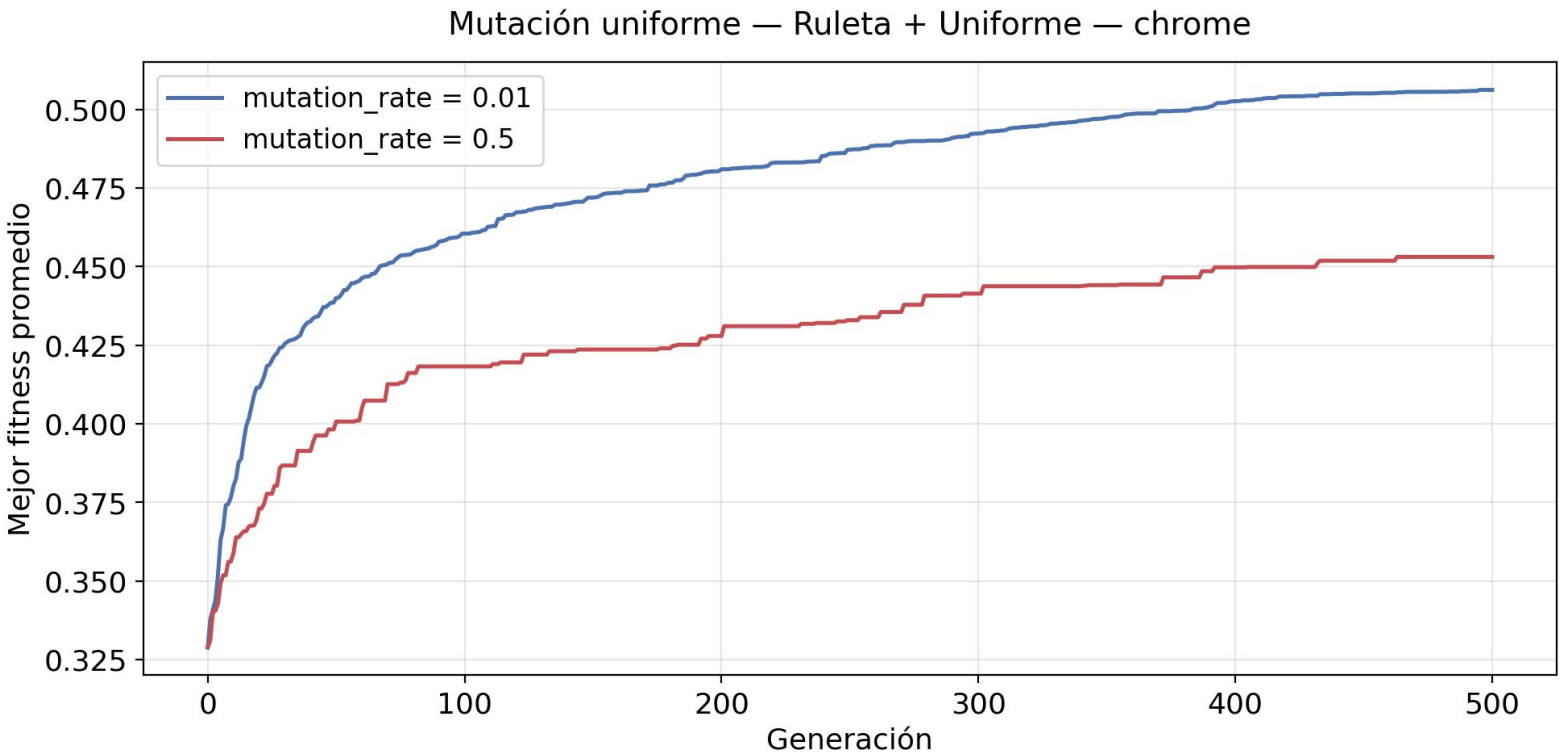
Objetivo

- Se evalúa el efecto de **mutation_rate**
- Dos métodos de **mutación**:
  - Mutación uniforme
  - Mutación por gen
- Se comparan:
  - mutation_rate = 0.01
  - mutation_rate = 0.5
- 2 configuraciones:
  - Selección: Élite y ruleta
  - Cruza: Dos puntos y uniforme

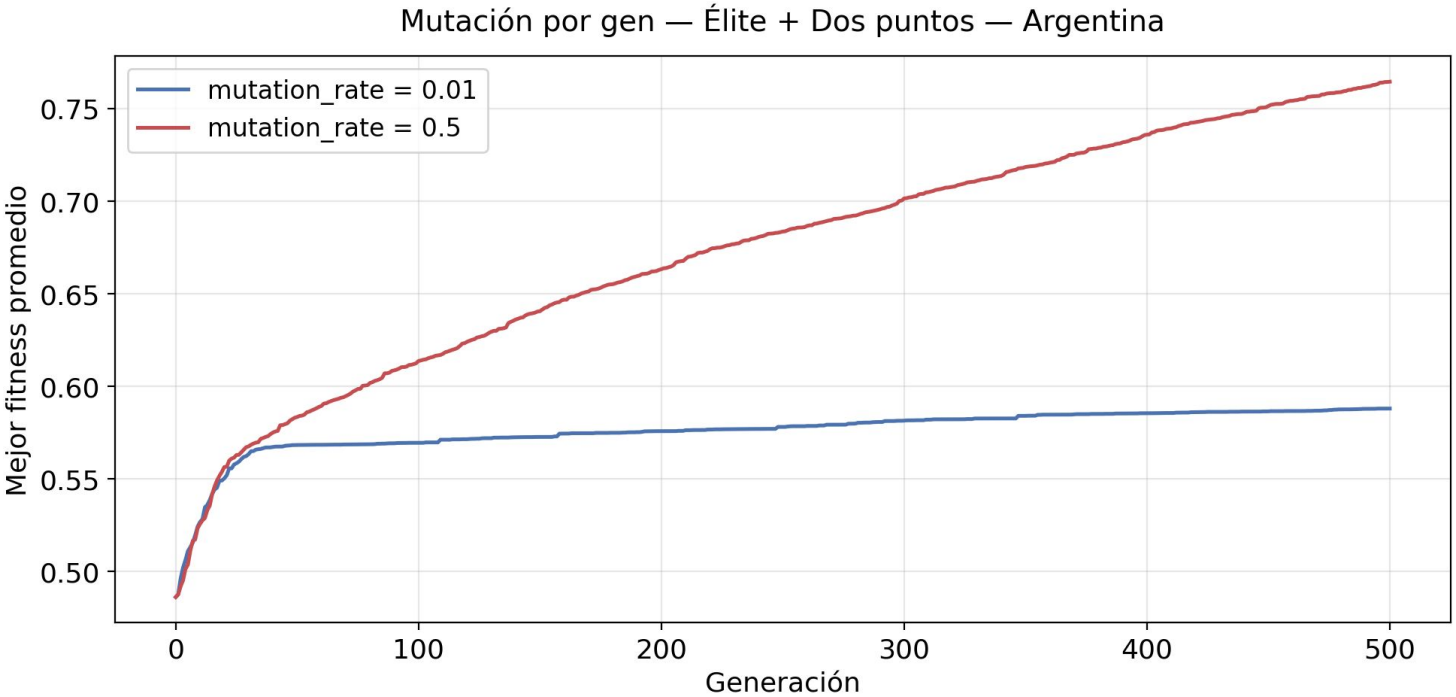
Resultados


Mutación uniforme — Élite + Dos puntos — Argentina
mutation_rate = 0.01
mutation_rate = 0.5
Mejor fitness promedio
0.650
0.625
0.600
0.575
0.550
0.525
0.500
0
100
200
300
400
500
Generación

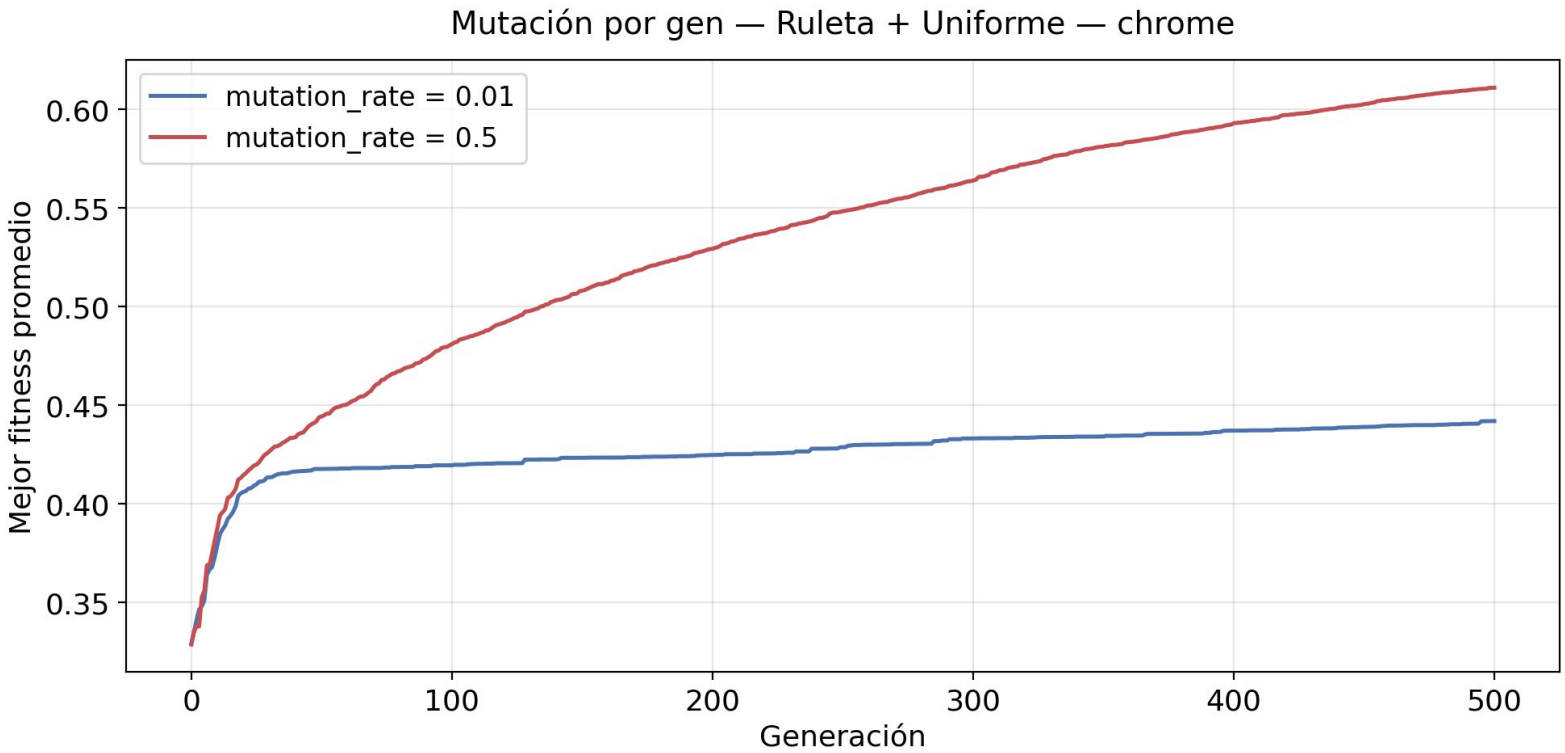
Resultados


Mutación uniforme — Ruleta + Uniforme — chrome
mutation_rate = 0.01
mutation_rate = 0.5
Mejor fitness promedio
0.500
0.475
0.450
0.425
0.400
0.375
0.350
0.325
0
100
200
300
400
500
Generación

Resultados


Mutación por gen — Élite + Dos puntos — Argentina
mutation_rate = 0.01
mutation_rate = 0.5
Mejor fitness promedio
0.75
0.70
0.65
0.60
0.55
0.50
0
100
200
300
400
500
Generación

Resultados


Mutación por gen — Ruleta + Uniforme — chrome
mutation_rate = 0.01
mutation_rate = 0.5
Mejor fitness promedio
0.60
0.55
0.50
0.45
0.40
0.35
0
100
200
300
400
500
Generación

Observaciones

- Mutación uniforme:
  - tasa alta → peor fitness final
  - convergencia más temprana
- Mutación por gen:
  - tasa alta → mejor fitness final
  - mejora sostenida
- El efecto de mutation_rate depende del metodo de mutación

Conclusiones

Conclusiones

- El rendimiento de los algoritmos genéticos depende fuertemente de la combinación de operadores
- No se puede optimizar cada componente de forma aislada
- El efecto de la probabilidad de mutación depende del método que se utilice

Gracias por su atención